

Kruskal's Algorithm

1. Problem Statement -

We are given a connected graph with a set of vertices and a set of edges. Each edge has a weight attached to it. The task is to find a Minimum Spanning Tree in this graph. A Minimum Spanning Tree is a smaller connected graph that touches every vertex once, has no cycle in it, and has the least possible total edge weight among all such trees.

Kruskal's algorithm is one of the methods used to solve this problem. It builds the spanning tree by picking edges one at a time in increasing order of their weight and adding an edge only if it does not form a cycle with the edges already chosen.

2. Approach Used -

Step 1: List all the edges of the graph along with their weights.

Step 2: Sort all the edges in increasing order of weight.

Step 3: Take the smallest edge that has not been checked yet.

Step 4: Check if adding this edge forms a cycle with the edges already selected. This is done using the union find method, also called disjoint set.

Step 5: If the edge does not form a cycle, add it to the spanning tree and combine the two sets it connects. If it forms a cycle, skip this edge.

Step 6: Repeat step 3 to step 5 until the spanning tree contains one less edge than the number of vertices.

3. C Program -

The following program implements Kruskal's algorithm using the steps explained above. It sorts the edges, then uses the union find method with path compression and rank to build the Minimum Spanning Tree. The program does not use any comments.

```
#include <stdio.h>
```

```
typedef struct {
```

```
    int src, dest, weight;
```

```
} Edge;
```

```
int parent[6];
```

```
int rank_[6];
```

```
int find(int x) {
```

```
    if (parent[x] != x)
```

```
        parent[x] = find(parent[x]);
```

```
    return parent[x];
```

```
}
```

```
int unionSet(int x, int y) {
```

```
    int rx = find(x);
```

```
    int ry = find(y);
```

```
    if (rx == ry)
```

```
        return 0;
```

```
    if (rank_[rx] < rank_[ry]) {
```

```
        parent[rx] = ry;
```

```
    }
```

```
    else if (rank_[rx] > rank_[ry]) {
```

```

        parent[ry] = rx;
    }
    else {
        parent[ry] = rx;
        rank_[rx]++;
    }

    return 1;
}

```

```

int sortEdges(Edge arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j].weight > arr[j + 1].weight) {
                Edge temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    return 0;
}

```

```

int main() {
    int V = 6;
    int E = 9;

```

```
Edge edges[9];
```

```
edges[0] = (Edge){0, 1, 4};
```

```
edges[1] = (Edge){0, 2, 4};
```

```
edges[2] = (Edge){1, 2, 2};
```

```
edges[3] = (Edge){2, 3, 3};
```

```
edges[4] = (Edge){2, 5, 2};
```

```
edges[5] = (Edge){2, 0, 4};
```

```
edges[6] = (Edge){2, 1, 3};
```

```
edges[7] = (Edge){2, 3, 2};
```

```
edges[8] = (Edge){2, 4, 4};
```

```
sortEdges(edges, E);
```

```
for (int i = 0; i < V; i++) {
```

```
    parent[i] = i;
```

```
    rank_[i] = 0;
```

```
}
```

```
Edge result[5];
```

```
int e = 0;
```

```
int i = 0;
```

```
int totalWeight = 0;
```

```
while (e < V - 1 && i < E) {
```

```
Edge nextEdge = edges[i++];

int x = find(nextEdge.src);
int y = find(nextEdge.dest);

if (x != y) {
    result[e++] = nextEdge;
    unionSet(x, y);
    totalWeight += nextEdge.weight;
}
}

printf("Edges in the Minimum Spanning Tree:\n");

for (i = 0; i < e; i++) {
    printf("%d -- %d == %d\n",
        result[i].src,
        result[i].dest,
        result[i].weight);
}

printf("Total weight of MST: %d\n", totalWeight);

return 0;
}
```

4. Output

The program was compiled and run on a graph with six vertices and nine edges. The output below shows the edges selected for the Minimum Spanning Tree along with their weights, and the total weight of the tree.

Edges in the Minimum Spanning Tree:

1 -- 2 == 2

2 -- 5 == 2

2 -- 3 == 3

0 -- 1 == 4

2 -- 4 == 4

Total weight of MST: 15